Campus Convenience Store POS

BSIT TODO | Exam of IT415

Proponents:
Morales, Cyrel Jave M.
Lariosa, Syclops

GitHub Repository:
https://github.com/Ajjime/Scenario-5-Campus-Convenience-Store-POS

Assigned Scenario:

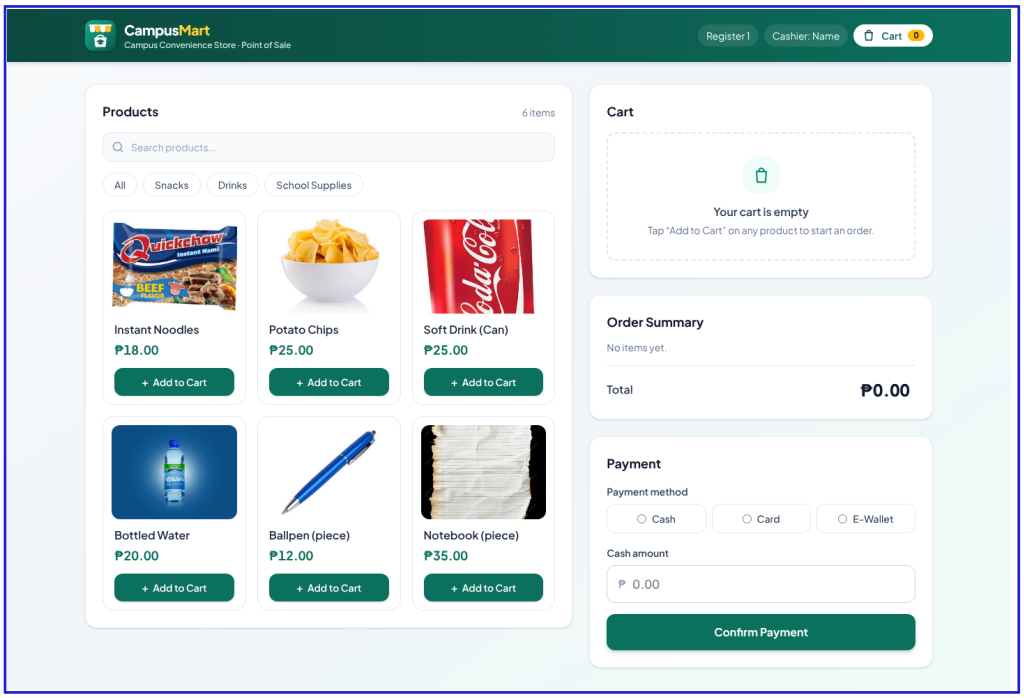
Scenario 5 — Campus Convenience Store POS

A small convenience store counter selling snacks, drinks, and basic school supplies.

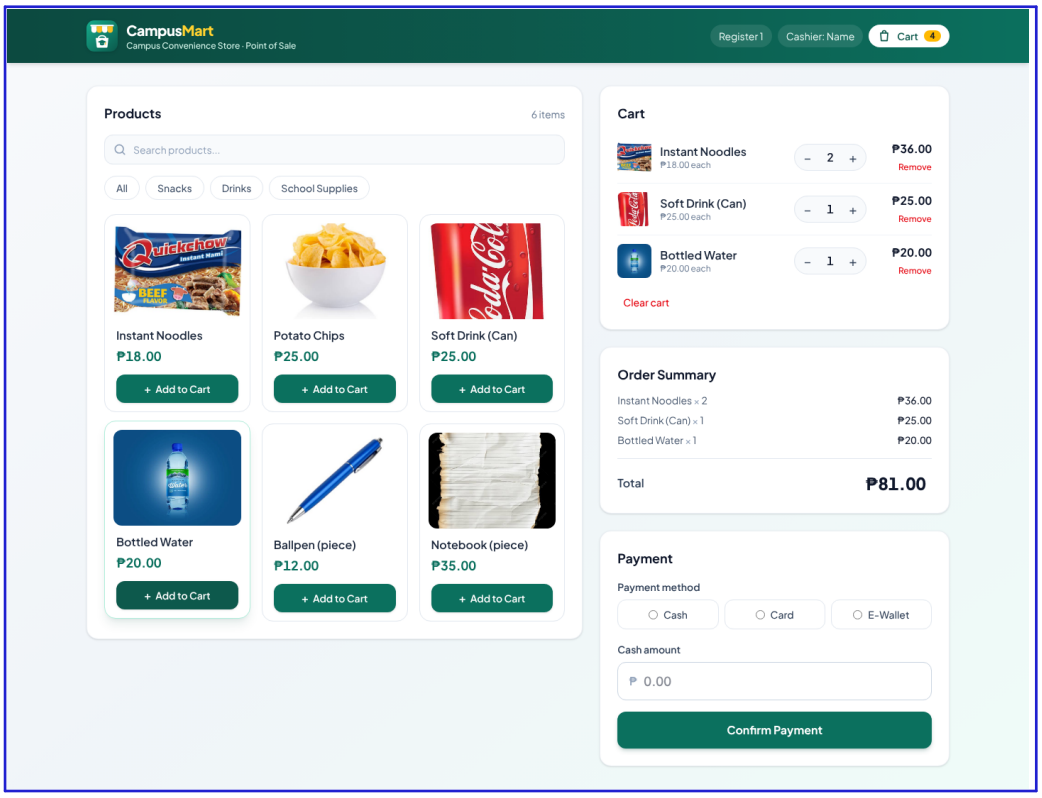
Product	Price
Instant Noodles	₱18.00
Potato Chips	₱25.00
Soft Drink (Can)	₱25.00
Bottled Water	₱20.00
Ballpen (piece)	₱12.00
Notebook (piece)	₱35.00

Screenshots of your working application (product list, cart, payment, and receipt) and of your GitHub commit history.

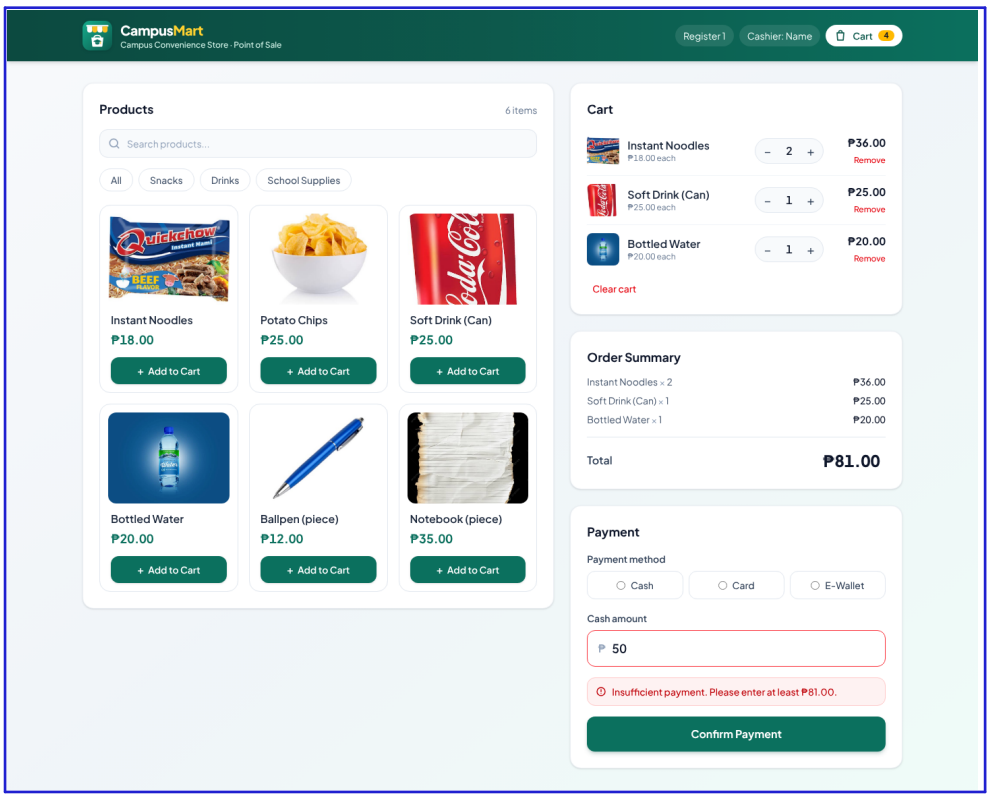
Product list present in the MENU.



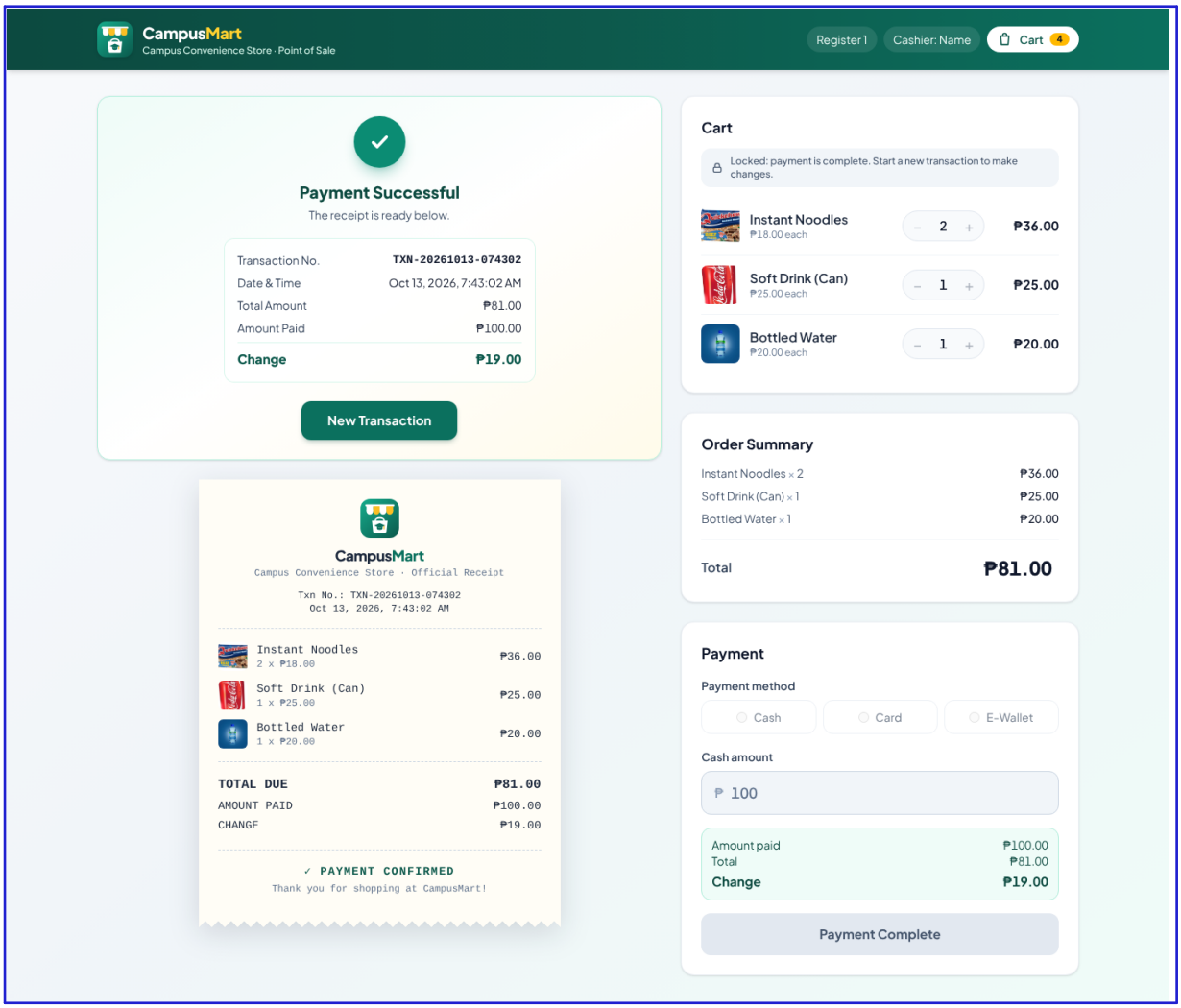
Cart and Order Summary. Subtotals and total are computed automatically.



Payment (Error Handling): insufficient payment.

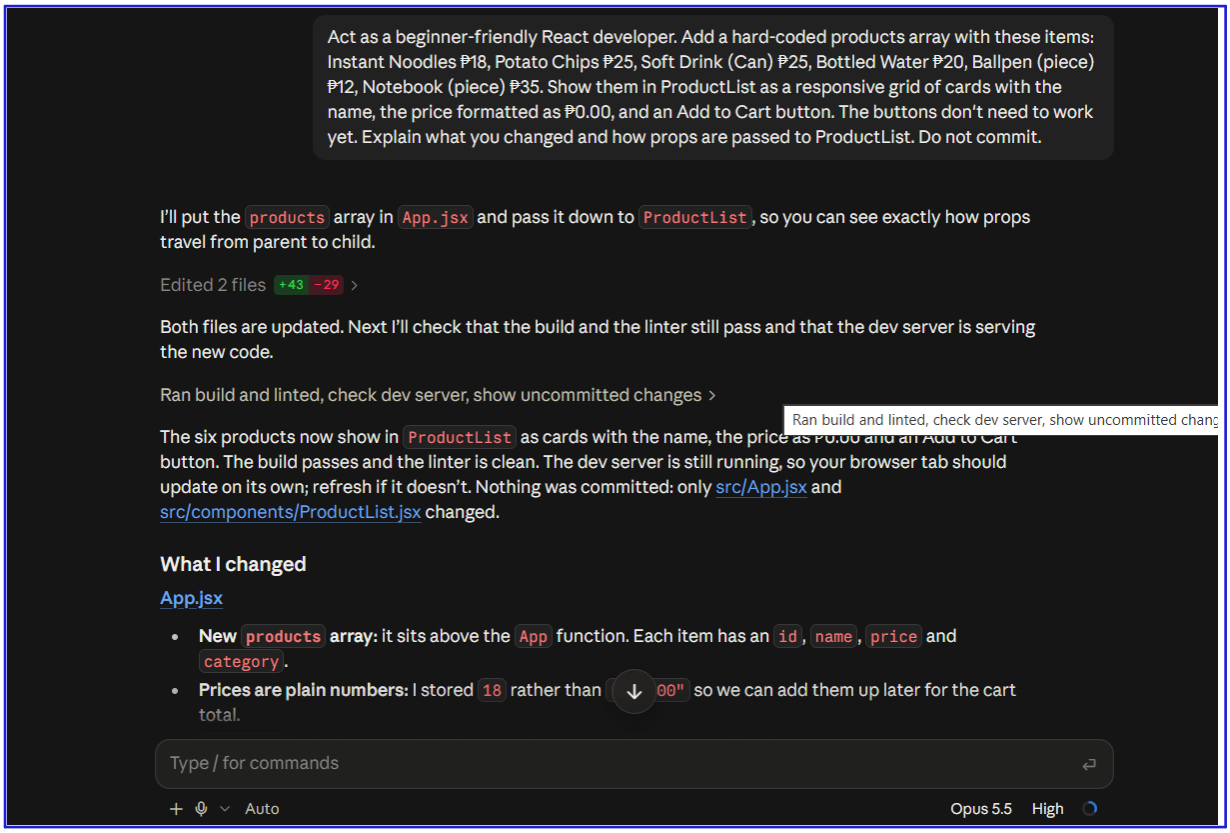


Payment confirmation and Receipt (Overall Total Summary).

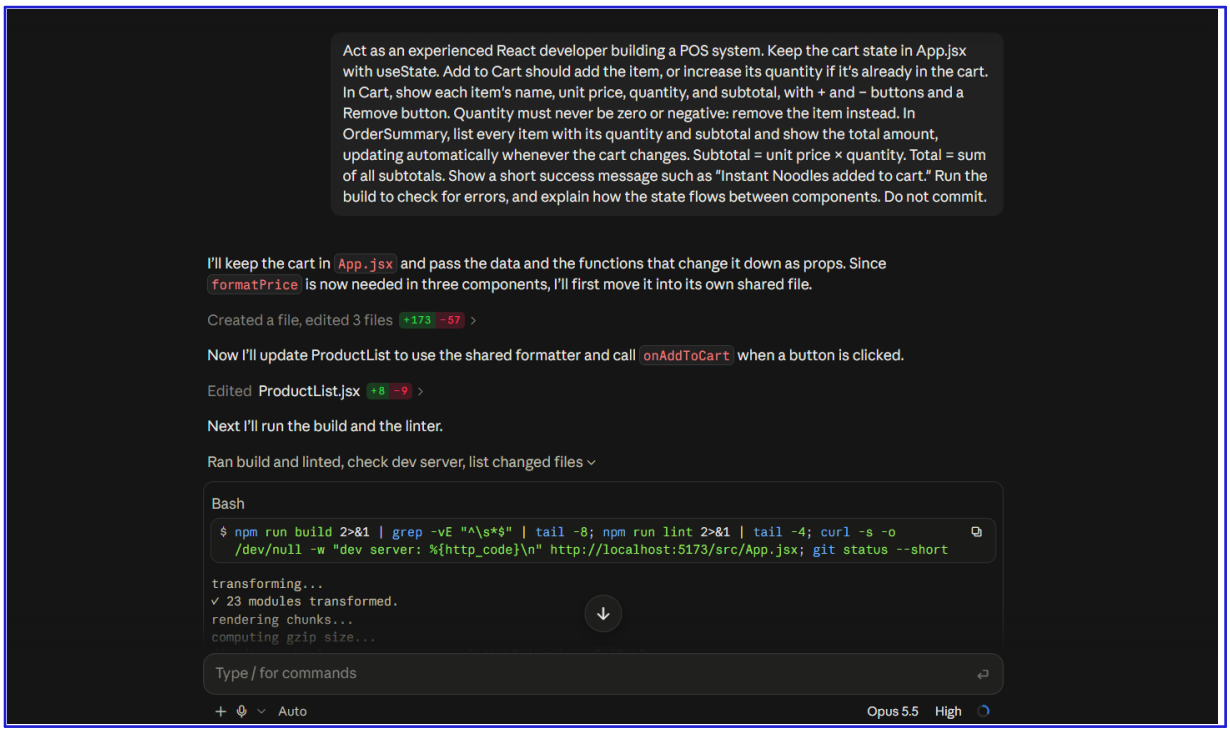


AI Prompt and Output Evidence

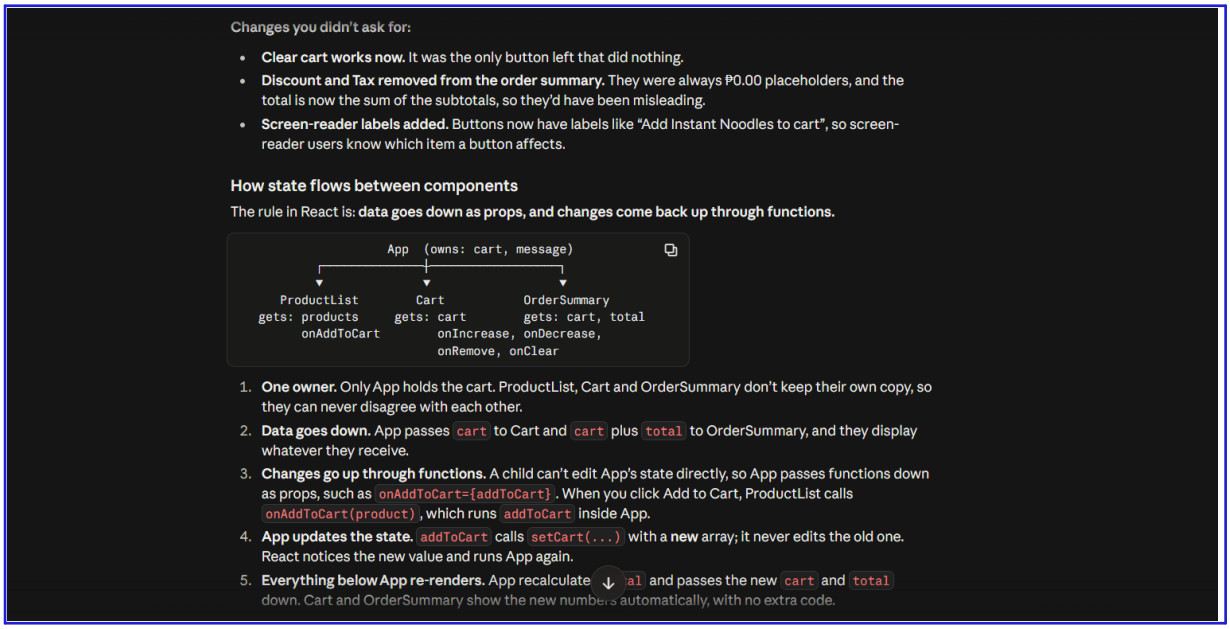
Prompt 1: Product list



Prompt 2: Cart, quantities, order summary



Prompt 2 (continued): how state flows between components



Changes you didn't ask for:

- **Clear cart works now.** It was the only button left that did nothing.
- **Discount and Tax removed from the order summary.** They were always ₱0.00 placeholders, and the total is now the sum of the subtotals, so they'd have been misleading.
- **Screen-reader labels added.** Buttons now have labels like "Add Instant Noodles to cart", so screen-reader users know which item a button affects.

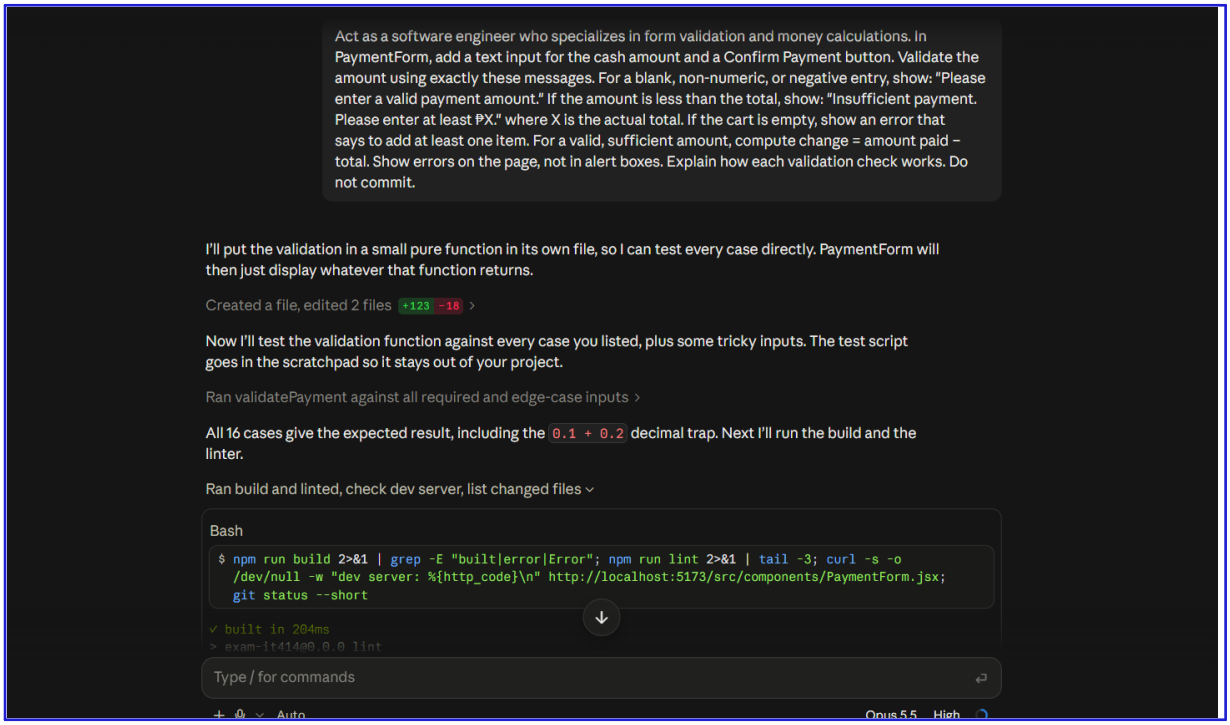
How state flows between components

The rule in React is: **data goes down as props, and changes come back up through functions.**

```
                App  (owns: cart, message)
                ┌──────────────┼──────────────┐
                ▼              ▼              ▼
   ProductList          Cart           OrderSummary
gets: products     gets: cart       gets: cart, total
     onAddToCart          onIncrease, onDecrease,
                          onRemove, onClear
```

1. **One owner.** Only App holds the cart. ProductList, Cart and OrderSummary don't keep their own copy, so they can never disagree with each other.
2. **Data goes down.** App passes `cart` to Cart and `cart` plus `total` to OrderSummary, and they display whatever they receive.
3. **Changes go up through functions.** A child can't edit App's state directly, so App passes functions down as props, such as `onAddToCart={addToCart}`. When you click Add to Cart, ProductList calls `onAddToCart(product)`, which runs `addToCart` inside App.
4. **App updates the state.** `addToCart` calls `setCart(...)` with a **new** array; it never edits the old one. React notices the new value and runs App again.
5. **Everything below App re-renders.** App recalculate `al` and passes the new `cart` and `total` down. Cart and OrderSummary show the new numbers automatically, with no extra code.

Prompt 3: Payment form and validation



Act as a software engineer who specializes in form validation and money calculations. In PaymentForm, add a text input for the cash amount and a Confirm Payment button. Validate the amount using exactly these messages. For a blank, non-numeric, or negative entry, show: "Please enter a valid payment amount." If the amount is less than the total, show: "Insufficient payment. Please enter at least ₱X." where X is the actual total. If the cart is empty, show an error that says to add at least one item. For a valid, sufficient amount, compute change = amount paid − total. Show errors on the page, not in alert boxes. Explain how each validation check works. Do not commit.

I'll put the validation in a small pure function in its own file, so I can test every case directly. PaymentForm will then just display whatever that function returns.

Created a file, edited 2 files +123 -18 ›

Now I'll test the validation function against every case you listed, plus some tricky inputs. The test script goes in the scratchpad so it stays out of your project.

Ran validatePayment against all required and edge-case inputs ›

All 16 cases give the expected result, including the `0.1 + 0.2` decimal trap. Next I'll run the build and the linter.

Ran build and linted, check dev server, list changed files ⌄

Bash

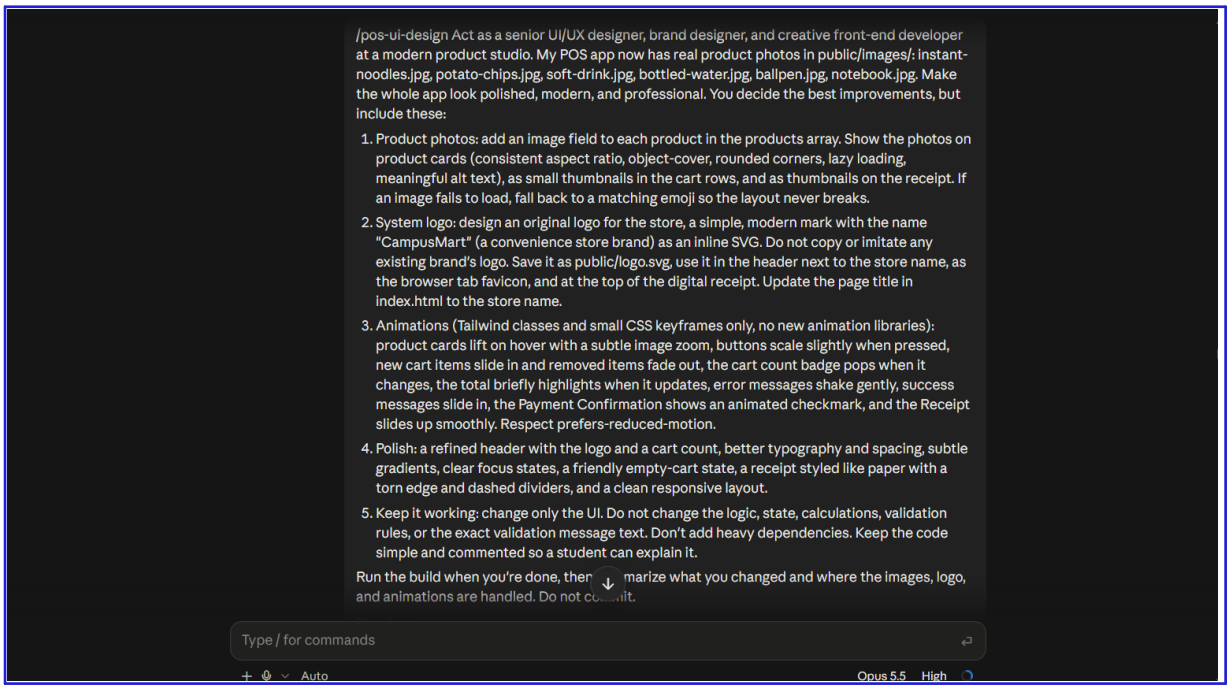
```
$ npm run build 2>&1 | grep -E "built|error|Error"; npm run lint 2>&1 | tail -3; curl -s -o
  /dev/null -w "dev server: %{http_code}\n" http://localhost:5173/src/components/PaymentForm.jsx;
  git status --short
```

```
✓ built in 204ms
> exam-it414@0.0.0 lint
```

Type / for commands

+ Auto Opus 5.5 High

Prompt 4: UI/UX polish (photos, logo, animations)



Your AI-assisted development documentation (Section VIII).

AI Prompt Record #1 (Building the product list)

Prompt Used: "Act as a beginner-friendly React developer. Add a hard-coded products array with these items: Instant Noodles ₱18, Potato Chips ₱25, Soft Drink (Can) ₱25, Bottled Water ₱20, Ballpen (piece) ₱12, Notebook (piece) ₱35. Show them in ProductList as a responsive grid of cards with the name, the price formatted as ₱0.00, and an Add to Cart button. The buttons don't need to work yet. Explain what you changed and how props are passed to ProductList. Do not commit."

Purpose of the Prompt: Display the six Scenario 5 products with names and prices (Required Feature 1) and show how props pass from App to ProductList.

AI Response Summary: The AI put a products array in App.jsx and passed it to ProductList as a prop. It edited two files (App.jsx and ProductList.jsx), ran the build and linter (both passed), and explained how data travels from parent to child. Prices were stored as plain numbers (18, not "₱18.00") so they can be added for the cart total.

Your Evaluation: Correct. The six products appear as cards with name, price formatted as ₱0.00, and an Add to Cart button, and the build and linter passed.

Modifications You Made: The buttons were left inactive on purpose, as requested, and were connected to the cart in the next prompt.

AI Prompt Record #2 (Building the cart and order summary)

Prompt Used: "Act as an experienced React developer building a POS system. Keep the cart state in App.jsx with useState. Add to Cart should add the item, or increase its quantity if it's already in the cart. In Cart, show each item's name, unit price, quantity, and subtotal, with + and − buttons and a Remove button. Quantity must never be zero or negative: remove the item instead. In OrderSummary, list every item with its quantity and subtotal and show the total amount, updating automatically whenever the cart changes. Subtotal = unit price × quantity. Total = sum of all subtotals. Show a short success message such as "Instant Noodles added to cart." Run the build to check for errors, and explain how the state flows between components."

Purpose of the Prompt: Implement the core cart logic: add, increase, decrease, remove, automatic subtotals and total, and the success message (Required Features 2 to 6).

AI Response Summary: The AI kept the cart in App.jsx, moved formatPrice into a shared file, and passed data and functions down as props. It created one file and edited four (+173/-57 lines in the first pass), then ran the build and linter. It explained the rule "data goes down as props, changes come back up through functions" with a component diagram. It also reported changes that were not requested: Clear cart now works, the placeholder Discount and Tax rows were removed, and screen-reader labels were added.

Your Evaluation: Correct. Adding, increasing, decreasing (removing at zero), and removing items all work, and the total updates automatically. The extra changes were

reasonable: the Discount and Tax rows were always ₱0.00 placeholders and would have been misleading.

Modifications You Made: Reviewed the AI's unrequested changes and kept them.

AI Prompt Record #3 (Payment form and validation)

Prompt Used: "Act as a software engineer who specializes in form validation and money calculations. In PaymentForm, add a text input for the cash amount and a Confirm Payment button. Validate the amount using exactly these messages. For a blank, non-numeric, or negative entry, show: "Please enter a valid payment amount." If the amount is less than the total, show: "Insufficient payment. Please enter at least ₱X." where X is the actual total. If the cart is empty, show an error that says to add at least one item. For a valid, sufficient amount, compute change = amount paid − total. Show errors on the page, not in alert boxes. Explain how each validation check works. Do not commit."

Purpose of the Prompt: Build payment entry, validation, and change computation (Required Features 7 to 9) using the exact messages from the exam.

AI Response Summary: The AI put the validation in a small standalone function (validatePayment.js) so every case can be tested directly, and PaymentForm only displays what it returns. It tested the function against 16 cases, including the 0.1 + 0.2 decimal trap, then ran the build and linter. Money is calculated in whole centavos.

Your Evaluation: Partially correct at first. All required cases worked (blank, non-numeric, negative, insufficient, empty cart, correct change). A later review found two gaps: "-0" was accepted because Number("-0") equals 0, and extremely large amounts could break the centavo math.

Modifications You Made: In the debugging commit (fcc3ba2) the negative check was changed to look at the minus sign itself (text.startsWith("-")), and a MAX_PAYMENT limit of ₱1,000,000 was added. The checks were renumbered and commented. I then re-ran 11 test inputs: blank, abc, -50, 100, 164.99, 165, 200, " 200 ", 20.555, 1e3, and an empty cart. All gave the expected result.

AI Prompt Record #4 (UI/UX polish: photos, logo, and animations)

Prompt Used: "/pos-ui-design Act as a senior UI/UX designer, brand designer, and creative front-end developer at a modern product studio. My POS app now has real product photos in public/images/: instant-noodles.jpg, potato-chips.jpg, soft-drink.jpg, bottled-water.jpg, ballpen.jpg, notebook.jpg. Make the whole app look polished, modern, and professional. You decide the best improvements, but include these: (1) Product photos on product cards, as thumbnails in the cart, and on the receipt, with an emoji fallback if an image fails to load. (2) An original CampusMart logo as an inline SVG saved as public/logo.svg, used in the header, as the favicon, and on the receipt; update the page title. (3) Animations with Tailwind classes and small CSS keyframes only: card hover lift, button press, cart item slide-in and fade-out, cart count pop, total highlight, error shake, success slide-in, animated checkmark, receipt slide-up; respect prefers-reduced-motion. (4) Polish: refined header, typography and spacing, gradients, focus states, empty-cart state, a receipt styled like paper with a torn edge and dashed dividers, and a responsive layout. (5) Keep it working: change only the UI. Do not change the logic, state, calculations, validation rules, or the exact validation message text. Do not add heavy dependencies. Keep the code simple and commented so a student can explain it. Run the build when you're done, then summarize what you changed and where the images, logo, and animations are handled. Do not commit it."

Purpose of the Prompt: Improve the look of the finished app (product photos, store logo, animations) without changing its working logic.

AI Response Summary: The AI added a ProductImage component with an emoji fallback, created public/logo.svg and used it in the header and receipt, added animations (hover lift, slide-in, pop, shake, checkmark, receipt slide-up) with CSS keyframes that respect reduced motion, and restyled the header, cart, and receipt. Logic, calculations, and validation messages were left unchanged.

Your Evaluation: Correct. The interface looks polished and everything still works: the full transaction flow and all validation messages behaved the same as before the redesign.

Modifications You Made: The product photos were added to public/images/ before this prompt. The Instant Noodles photo was added later, in the bug-fix commit (fcc3ba2).

GitHub Commit History

